Global Institute of Technology Jaipur, Rajasthan	
Notes	
Department of Computer Science & Engineering	
3CS4-05 Data Structures and Algorithms -Total Lecture Required:	
Faculty Name : Asst.Professor Vaishali Sharma	**IA(MT1+MT2+Assignment) : 30 Marks**
B.Tech. II Yr.- III Sem C	**ETE : 70 Marks**
Session : Odd Semester 2022-23	**Exam Duration : 3Hrs**
Commencment of Semester: 08/08/2022	**End of Semester : 30/11/2022**

COURSE OUTCOMES:

The study of subject **Data Structures and Algorithms 3CS4-05** in undergraduate program in Computer Science Engineering Branch will achieve the following major objective-

1. This subject helps the student to understand concept of Stack Data Structure, its applications and uses with various algorithms.
2. This subject helps the student to understand another concept of Queue & Linked List Data Structure, its applications and uses with various algorithms.
3. This subject helps the student to design algorithms using different sorting techniques to sort various numbers by various methods.
4. This subject helps the student to understand concept of Tree Data Structure to store data in computer, its applications.
5. This subject helps the student to understand concept of Graph Data Structure to find shortest path to reach from one point to another point and Hashing concept.

UNIT – I Stack DATA STRUCTURES

Basic Concepts: Introduction to Data Structures:

A data structure is a way of storing data in a computer so that it can be used efficiently and it will allow the most efficient algorithm to be used. A well-designed data structure allows a variety of critical operations to be performed, using as few resources, both execution time and memory space, as possible. Data structure introduction refers to a scheme for organizing data, or in other words it is an arrangement of data in computer's memory in such a way that it could make the data quickly available to the processor for required calculations.

A data structure should be seen as a logical concept that must address two fundamental concerns.

1. First, how the data will be stored, and
2. Second, what operations will be performed on it.

Classification of Data Structures:

Data structures can be classified as

- Linear data structure
- Non-linear data structure

Linear Data Structure:

Linear data structures can be constructed as a continuous arrangement of data elements in the memory. It can be constructed by using array data type. In the linear Data Structures the relationship of adjacency is maintained between the data elements.

Operations applied on linear data structure:
The following list of operations applied on linear data structures

1. Add an element
2. Delete an element
3. Traverse
4. Sort the list of elements
5. Search for a data element

For example Stack, Queue, Array and Linked Lists.

Non-linear Data Structure:

Non-linear data structure can be constructed as a collection of randomly distributed set of data item joined together by using a special pointer (tag). In non-linear Data structure the relationship of adjacency is not maintained between the data items.

Operations applied on non-linear data structures:
The following list of operations applied on non-linear data structures.

1. Add elements
2. Delete elements
3. Display the elements
4. Sort the list of elements
5. Search for a data element

For example Tree, Decision tree, Graph and Forest

Algorithms:

Structure and Properties of Algorithm:

An algorithm has the following structure

1. Input Step
2. Assignment Step
3. Decision Step
4. Repetitive Step
5. Output Step

An algorithm is endowed with the following properties:

1. **Finiteness:** An algorithm must terminate after a finite number of steps.
2. **Definiteness:** The steps of the algorithm must be precisely defined or unambiguously specified.
3. **Generality:** An algorithm must be generic enough to solve all problems of a particular class.
4. **Effectiveness:** the operations of the algorithm must be basic enough to be put down on pencil and paper. They should not be too complex to warrant writing another algorithm for the operation.
5. **Input-Output:** The algorithm must have certain initial and precise inputs, and outputs that may be generated both at its intermediate and final steps.

Different Approaches to Design an Algorithm:

An algorithm does not enforce a language or mode for its expression but only demands adherence to its properties.

Practical Algorithm Design Issues:

1. **To save time (Time Complexity):** A program that runs faster is a better program.
2. **To save space (Space Complexity):** A program that saves space over a competing program is considerable desirable.

Efficiency of Algorithms:

The performances of algorithms can be measured on the scales of **time** and **space**. The performance of a program is the amount of computer memory and time needed to run a program. We use two approaches to determine the performance of a program.

Time Complexity: The time complexity of an algorithm or a program is a function of the running time of the algorithm or a program. In other words, it is the amount of computer time it needs to run to completion.

Space Complexity: The space complexity of an algorithm or program is a function of the space needed by the algorithm or program to run to completion.

LINEAR DATA STRUCTURES

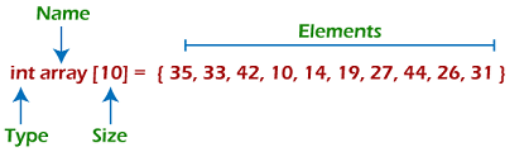
Array in Data Structure

Arrays are defined as the collection of similar types of data items stored at contiguous memory locations. It is one of the simplest data structures where each data element can be randomly accessed by using its index number.

There are some of the properties of an array that are listed as follows -

- Each element in an array is of the same data type and carries the same size that is 4 bytes and the size of the short type is 2 bytes (16 bits).
- Elements in the array are stored at contiguous memory locations from which the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

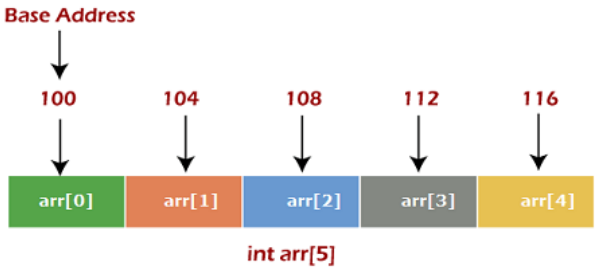
Representation of an array



Arrays are useful because -

- Sorting and searching a value in an array is easier.
- Arrays are best to process multiple values quickly and easily.
- Arrays are good for storing multiple values in a single variable

Memory allocation of an array



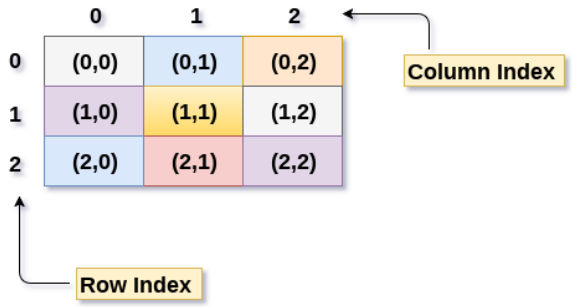
Basic operations

Now, let's discuss the basic operations supported in the array -

- Traversal - This operation is used to print the elements of the array.
- Insertion - It is used to add an element at a particular index.
- Deletion - It is used to delete an element from a particular index.
- Search - It is used to search an element using the given index or by the value.
- Update - It updates an element at a particular index.

2D Array

2D array can be defined as an array of arrays.



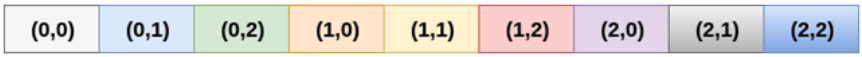
Declaration of 2D Array

```
int arr[max_rows][max_columns];
```

There are two main techniques of storing 2D array elements into memory:-

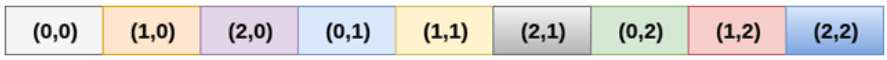
1. Row Major ordering

In row major ordering, all the rows of the 2D array are stored into the memory contiguously.



2. Column Major ordering

According to the column major ordering, all the columns of the 2D array are stored into the memory contiguously.



Calculating the Address of the random element of a 2D array

Row Major

*Address(a[i][j]) = B. A. + (i * n + j) * size*

a[10...30, 55...75], base address of the array (BA) = 0, size of an element = 4 bytes .

Find the location of a[15][68].

Address(a[15][68]) = 0 + ((15 - 10) x (68 - 55 + 1) + (68 - 55)) x 4

= (5 x 14 + 13) x 4

= 83 x 4

= 332 answer

Column Major

*Address(a[i][j]) = ((j*m)+i)*Size + BA*

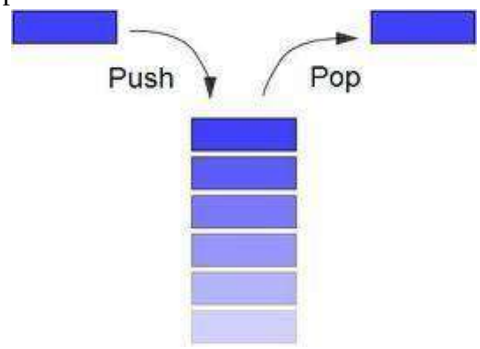
A [-5 ... +20][20 ... 70], BA = 1020, Size of element = 8 bytes. Find the location of a[0][30].

Address [A[0][30]) = ((30-20) x 24 + 5) x 8 + 1020 = 245 x 8 + 1020 = 2980 bytes

Stack:

A stack is a container of objects that are inserted and removed according to the last-in first-out (LIFO) principle. In the pushdown stacks only two operations are allowed: push the item into the stack, and pop the item out of the stack. A stack is a limited access data structure - elements can be added and removed from the stack only at the top. Push adds an item to the top of the stack, pop removes the item from the top. A helpful analogy is to think of a stack of books; you can remove only the top book, also you can add a new book on the top.

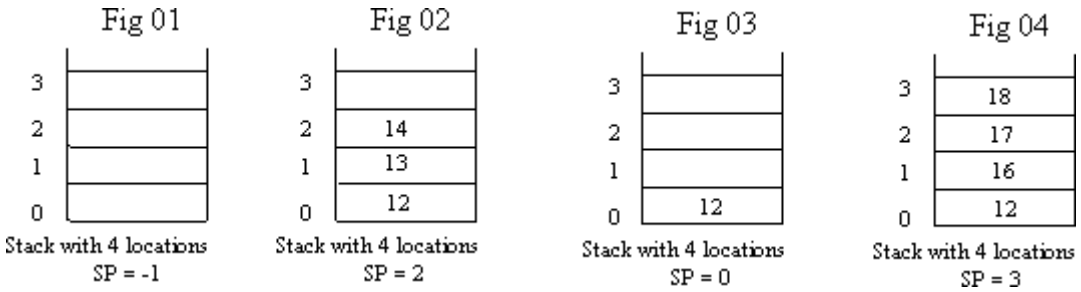
A stack may be implemented to have a bounded capacity. If the stack is full and does not contain enough space to accept an entity to be pushed, the stack is then considered to be in an overflow state. The pop operation removes an item from the top of the stack. A pop either reveals previously concealed items or results in an empty stack, but, if the stack is empty, it goes into underflow state, which means no items are present in stack to be removed.



Stack Data Structure:

Stack is an Abstract data structure works on the principle Last In First Out (LIFO). The last element add to the stack is the first element to be delete. Insertion and deletion can be takes place at one end called TOP. It looks like one side closed tube.

- The add operation of the stack is called push operation
- The delete operation is called as pop operation.
- Push operation on a full stack causes stack overflow.
- Pop operation on an empty stack causes stack underflow.
- SP is a pointer, which is used to access the top element of the stack.
- If you push elements that are added at the top of the stack;
- In the same way when we pop the elements, the element at the top of the stack is deleted.



Operations of stack:

There are two operations applied on stack they are

1. push
2. pop.

While performing push & pop operations the following test must be conducted on the stack.

1) Stack is empty or not

2) Stack is full or not

Push:

Push operation is used to add new elements in to the stack. At the time of addition first check the stack is full or not. If the stack is full it generates an error message "stack overflow".

Pop:

Pop operation is used to delete elements from the stack. At the time of deletion first check the stack is empty or not. If the stack is empty it generates an error message "stack underflow".

Representation of a Stack using Static Array:

Array based Implementation

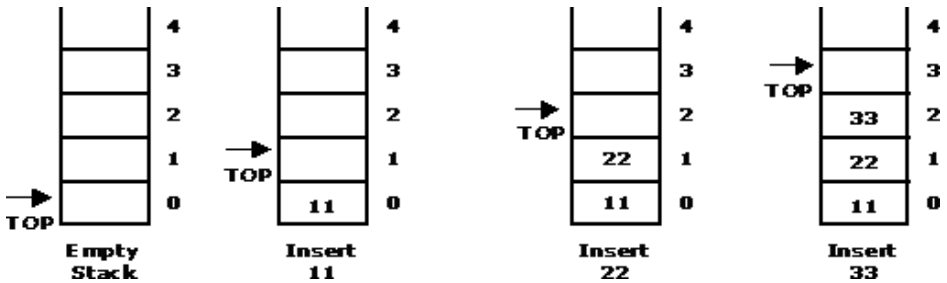
- Using Array or Static Array (Array size is fixed and has to be given during initialization)
- Using Dynamic Arrays or Resizable Arrays (Arrays can grow or shrink based on requirement)

Pop Operation Pseudocode

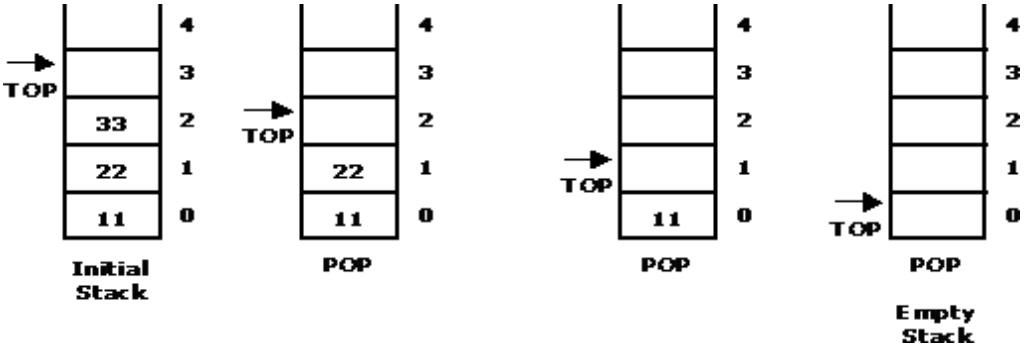
1. If stack is empty
   - Overflow
2. Else
   - remove the element at top index
   - update top index to top-1

Let us consider a stack with 6 elements capacity. This is called as the size of the stack. The number of elements to be added should not exceed the maximum size of the stack. If we attempt to add new element beyond the maximum size, we will encounter a stack overflow condition. Similarly, you cannot remove elements beyond the base of the stack. If such is the case, we will reach a stack underflow condition.

When a element is added to a stack, the operation is performed by push().



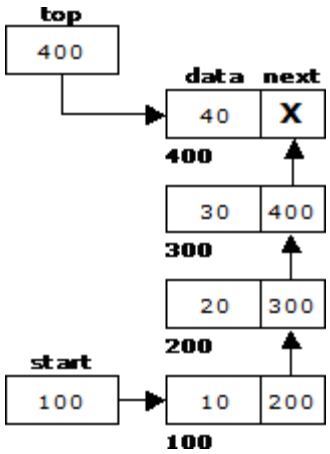
When an element is taken off from the stack, the operation is performed by pop().



STACK: Stack is a linear data structure which works under the principle of last in first out. Basic operations: push, pop, display.

1. **PUSH**: if (top==MAX), display **Stack overflow** else reading the data and making stack [top] =data and incrementing the top value by doing top++.
2. **POP:** if (top==0), display **Stack underflow** else printing the element at the top of the stack and decrementing the top value by doing the top.
3. **DISPLAY**: IF (TOP==0), display **Stack is empty** else printing the elements in the stack from stack [0] to stack [top].

We can represent a stack as a linked list. In a stack push and pop operations are performed at one end called top. We can perform similar operations at one end of list using top pointer.



Representation of a Stack using Dynamic Array:

Push Operation Pseudocode

1. If stack is full (If the stack has reaches its maximum capacity)

- resize the array

2. Else

- increment top by 1
- update stack[top] with given data

Implementing multiple Stacks in a Single Array:

When a stack is created using single array, we cannot able to store large amount of data, thus this problem is rectified using more than one stack in the same array of sufficient array.

To implement multiple stacks in a single array, one approach is to divide the array in k slots of size n/k each, and fix the slots for different stacks, we can use arr[0] to arr[n/k-1] for first stack, and arr[n/k] to arr[2n/k-1] for stack2 and so on where arr[] is the array of size n.

Although this method is easy to understand, but the problem with this method is inefficient use of array space.A stack push operation may result in stack overflow even if there is space available in arr[].

Stack Applications:

1. Stack is used by compilers to check for balancing of parentheses, brackets and braces.
2. Stack is used to evaluate a postfix expression.
3. Stack is used to convert an infix expression into postfix/prefix form.
4. In recursion, all intermediate arguments and return values are stored on the processor"s stack.
5. During a function call the return address and arguments are pushed onto a stack and on return they are popped off.
6. Depth first search uses a stack data structure to find an element from a graph.

Factorial(n)

Input: integer $n \geq 0$ Output: n!

If n = 0 then return (1)

else return prod(n, factorial(n − 1))

In-fix to Postfix Transformation:

Procedure:

Procedure to convert from infix expression to postfix expression is as follows:

Scan the infix expression from left to right.

a) If the scanned symbol is left parenthesis, push it onto the stack.

If the scanned symbol is an operand, then place directly in the postfix expression (output).

If the symbol scanned is a right parenthesis, then go on popping all the items from the stack and place them in the postfix expression till we get the matching left parenthesis.

If the scanned symbol is an operator, then go on removing all the operators from the stack and place them in the postfix expression, if and only if the precedence of the operator which is on the top of the stack is greater than (or equal) to the precedence of the scanned operator and push the scanned operator onto the stack otherwise, push the scanned operator onto the stack.

Convert the following infix expression A + B * C – D / E * H into its equivalent postfix expression.

Symbol	Postfix string	Stack	Remarks
A	A	(	
+	A	(+	
B	A B	(+	
*	A B	(+ *	
C	A B C	(-	
-	A B C * +	(-	
D	A B C * + D	(-	
/	A B C * + D	(- /	
E	A B C * + D E	(- /	

*	A B C * + D E /	(- *	
H	A B C * + D E / H	(- *	
End of string	A B C * + D E / H * -	The input is now empty. Pop the output symbols from the stack until it is empty.	

Evaluating Arithmetic Expressions:

Procedure:

The postfix expression is evaluated easily by the use of a stack. When a number is seen, it is pushed onto the stack; when an operator is seen, the operator is applied to the two numbers that are popped from the stack and the result is pushed onto the stack.

Evaluate the postfix expression: 6 5 2 3 + 8 * + 3 + *

Symbol	Operand 1	Operand 2	Value	Stack	Remarks
6				6	
5				6, 5	
2				6, 5, 2	
3				6, 5, 2, 3	The first four symbols are placed on the stack.
+	2	3	5	6, 5, 5	Next a „+" is read, so 3 and 2 are popped from the stack and their sum 5, is pushed
8	2	3	5	6, 5, 5, 8	Next 8 is pushed
*	5	8	40	6, 5, 40	Now a „*" is seen, so 8 and 5 are popped as 8 * 5 = 40 is pushed
+	5	40	45	6, 45	Next, a „+" is seen, so 40 and 5 are popped and 40 + 5 = 45 is pushed
3	5	40	45	6, 45, 3	Now, 3 is pushed
+	45	3	48	6, 48	Next, „+" pops 3 and 45 and pushes 45 + 3 = 48 is pushed
*	6	48	288	**288**	Finally, a „*" is seen and 48 and 6 are popped, the result 6 * 48 = 288 is pushed

Recursion:

In general terms recursion means the process to define a problem or the solution for a problem in a much simpler way compared to the original version. It is a problem-solving programming technique that has a remarkable and unique characteristic.

In recursion in data structure, a method or a function has the capability to decode an issue. In the process of recursion, a problem is resolved by transforming it into small variations of itself. In this procedure, the function can call itself either directly or indirectly. Based on the differences in call recursion in data structure can be categorized into different categories. You will get to know about these categories later in the blog.

Recursion in data structure is additionally an adequate programming technique. A recursive subroutine is described as one that directly or indirectly calls itself. Calling a subroutine specifically indicates that the characterization of the subroutine already has the call statement of calling the subroutine that has been defined.

Moreover, the indirect calling of the subroutine takes place whenever a subroutine calls another subroutine, which in turn calls the first subroutine. Recursion in data structure can use several lines of code to refer to an elaborate job.

What Is a Recursive Algorithm?

A recursive algorithm calls itself with small input values & returns the outcome for the present input by executing fundamental functions on the returned value for the small input. By and large, if a problem could be sorted out by using remedies to smaller adaptations of the same problem, in addition to the smaller variations reduced to conveniently solvable cases, then the problem could be fixed employing a recursive algorithm.

How does Recursion work?

Several programming languages provide recursive functions to call themselves directly or indirectly by calling another function that eventually ends up calling the initial function. The memory devoted anytime a function is called upon is once again relocated besides memory allocated to the calling function while duplicates of local variables are made for every single call.

As soon as the root issue has been developed, functions return their value to functions through which they are called. The moment this takes place, memories are de-allocated and the cycle repeats.

One crucial element of a recursive function is the serious need for a base instance or termination point. Every program created with recursion must make sure the functions are terminated, failing which could result in the system crashing errors.

5 Important Recursion in Data Structure Methods

There are 5 effective recursion techniques programmers can make use of in functional programming. And, they are

Tail Recursion in Data Structure

Even though linear recursion is by far the most widely used technique, tail recursion is considerably more beneficial. When you use the tail recursion process, the recursive functions are called in the end.

Binary Recursion in Data Structure

When working with binary recursion, functions are called upon 2 times rather than being called one by one. This specific form of recursion data structure is used in operations like merging and also tree traversal.

Linear Recursion in Data Structure

It is the most widely used recursion method. applying this approach, functions call themselves in a non-complex form and then terminate the function with a termination condition. This particular strategy consists of functions making one-off calls to itself during execution.

Mutual Recursion in Data Structure

It is the approach where a function will use others recursively. Functions turn out calling one another in this technique. This strategy is particularly used when writing programs making use of programming languages that often do not support recursive calling functions. Hence, implementing mutual recursion can serve as an alternative for a recursion function. In mutual recursion, starting circumstances are used for a single, several, or even all of the functions.

Types of Recursion in Data Structure

There are five types of recursion in data structure that are broadly categorized into two major types of recursion. These are direct recursion and indirect recursion.

Direct Recursion in Data Structure

In the direct recursion, functions call themselves. This kind of operation consists of a single-stage recursive call by the function from within itself. Why don’t we investigate precisely how to carry out direct recursion to determine the square root of a number.

```
{
  // base case
    if (x == 0)
    {
      return x;
    }
  // recursive case
  else
   {
     return square(x-1) + (2*x) – 1;
   }
}
int main() {
  // execution of square function
  int input=3;
  cout << input<<“^4 = “<<square(input);
  return 0;
}
```

The output would be displayed like this:

```
3^4 = 9
```

Indirect Recursion in Data Structure

Indirect recursion happens when functions call other functions to call the initial function. This specific course of action consists of 2 simple steps when developing a recursive call, essentially making functions call functions to generate a recursive call. Mutual recursion could be referred to as indirect recursion.

Let’s examine precisely how to put into action indirect recursion to print or perhaps find out all of the figures from 10 to 20.

```
using namespace std;
int n=10;
// declaring functions
void foo1(void);
void foo2(void);
// defining recursive functions
void foo1()
{
```

```
 if (n <= 20)
 {
   cout<<n<<" ";  // prints n
   n++;           // increments n by 1
   foo2();       // calls foo2()
 }
 else
   return;
}
void foo2()
{
 if (n <= 20)
 {
   cout<<n<<" ";  // prints n
   n++;           // increments n by 1
   foo1();       // calls foo1()
 }
 else
   return;
}
```

The output would be displayed like this:

10 11 12 13 14 15 16 17 18 19 20

Tailed Recursion in Data Structure

A recursive function is referred to as tail-recursive if the recursive call is the end execution executed by the function. Let us try to figure out the definition with the help of one example.

```
int fun(int z)
{
 printf("%d",z);
 fun(z-1);
 //Recursive call is last executed statement
}
```

If you notice this particular program, you can observe that the last line ADI is going to execute for method fun is a recursive call. And due to that, there's no need to recall any earlier state of the program.

Non-Tailed Recursion in Data Structure

A recursive function is said to be non-tail recursive in case the recursion call isn't the final thing performed by the function. After reverting back, there's another thing still there to assess. Now, look at the example.

```
int fun(int z){

  fun(z-1);

  printf("%d",z);  //Recursive call is not the last executed statement  }
```

In this particular function, you can notice that there is another operation following the recursive call. Hence the ADI is going to have to remember the preceding state within this specific method block. That's the reason this program could be regarded as non-tail recursive.

When is Recursion in Data Structure Used?

You'll find circumstances where you can take advantage of iteration or recursion in data structure. Nevertheless, you must always select a solution that seems to be the considerably more natural fit for a challenge. A recursion in data structure is, obviously, the right choice when we talk about data abstraction. Folks generally use recursive definitions to describe data along with associated operations.

It will not be inaccurate to imply that recursion in data structure is usually the natural alternative for issues related to the application of distinct operations on data. Nevertheless, you can get certain things related to recursion in data structure that could not make it the ideal answer for every situation. In these situations, a solution such as the iterative technique is the greatest fit.

The implementation of recursion in data structure uses a good deal of stack space, which may usually lead to redundancy. Whenever we make use of recursion in data structure, we call a method that results in the formation of a whole new instance of that approach. A new instance holds variables and parameters that are kept on the stack, and therefore are taken on the return. So while recursion in data structure is a slightly more straightforward option as opposed to others, it is not often the most practical.

Additionally, we do not have some predefined guidelines that may help select recursion or iteration for various issues. The most significant advantage of using recursion in data structure is that it's a concise technique. This process makes checking out and maintaining it much easier than usual. But recursive methods are not the most effective approaches available to us as they take a good deal of storage capacity and use up plenty of time during implementation.

Acknowledging a couple of factors can make it easier to determine if selecting a recursion in data structure for a problem is the appropriate way to go or not. You must opt for recursion if the issue that you're intending to fix is pointed out in recursive terms and the recursive solution appears to be less complicated.

You must remember that recursion in data structure, in nearly all cases, simplifies the implementation of the algorithms that you would like to use. However, if the complexities connected with making use of iteration and recursion are the same for a specified issue, you must opt for iteration as the prospects of it being a lot more effective are higher.

Difference between Recursion and Iteration

Criteria	**Recursion**	**Iteration**
Definition	When a function calls itself directly or indirectly	When there are some set of instructions that are carried out repeatedly
Implementation	Recursion is implemented with the use of function calls	Iteration is implemented by the use of loops
Memory Usage	Stack memory is used to store local variables & parameters	No memory is used except for the initialization of control variables

Speed	Slow speed of execution	Fast speed of execution
Code Size	Recursion code is usually smaller and simpler	Iterative code is usually big
Format	Recursive and base case association is specified	It includes initializing control variable, control variable update, and termination condition
Progression	The base case is approached by the function state	The termination value is approached by the control variable
Current State	It is defined by the parameters kept in the stack	It is defined by the value of control variables
Overhead	Includes overhead of repeated function call	As there are no function calls therefore no overhead
Repetition	If the base case is undefined or not reached then it will cause a stack overflow error	If the control variable is unable to reach termination value then it will cause an infinite loop

Using Recursion in Data Structure C++

Recursion in data structure is quite widely used in programming languages including C++. Let's examine exactly how to work with the recursive function in C++ to carry out recursion in data structure when creating a program to obtain the factorial of 6.

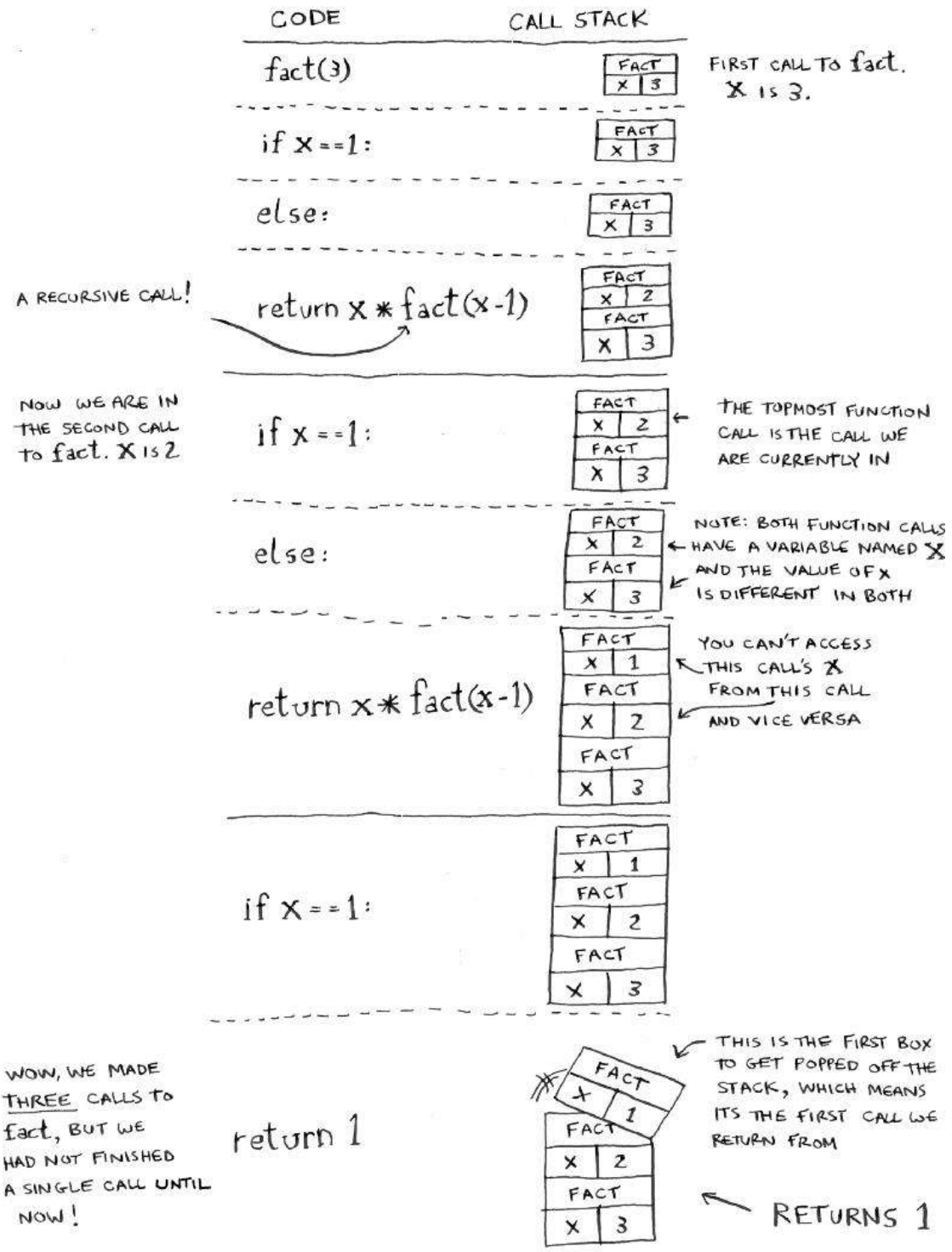
```
using namespace std;

// Factorial function
int f(int n)
{
   // Stop condition
   if ( n == 0 || n == 1 )
      return 1;
    // Recursive condition
   else
      return n * f(n – 1);
}
 // Driver code
int main()
{
   int n = 6;
   cout<<"Required factorial of 6 "<<n<<" = "<<f(n);
```

```
    return 0;
}
```

The output is displayed as:

Required factorial of 6 = 720



Recursion in Data Structure Example

Reversing an Array

Let's look at the issue of reversing the n components of an array, A so that the first component turns into the final, the 2nd component will become the 2nd to the last, and so forth. We can work out this specific challenge with the linear recursion in data

structure, by paying attention to how the reversal of an array could be done by swapping the first and last components and subsequently recursively reversing the rest of the components in the array.

Algorithm Reverse Array(A, i, j):

Input: An array A & non-negative integer indices i & j

Output: Reversal of the elements in A starting from index i & ending at j

```
if i < j then
    Swap A[i] and A[j]
    ReverseArray(A, i+1, j-1)
return
```

Fibonacci Sequence

Fibonacci sequences are the sequence of numbers 1, 1, 2, 3, 5, 8, 13, 21, 34, 55…. The initial two numbers of the sequence are the two 1, while every succeeding number is the sum of the two numbers prior to it. We can set a function F(n) that will calculate the nth Fibonacci number.

Algorithm Linear Fib(n) {

Input: A nonnegative integer n

Output: Pair of Fibonacci numbers (Fn, Fn-1)

```
if (n <= 1) then
    return (n, 0)
else
    (i, j) <– LinearFib(n-1)
    return (i + j, i)
}
```

Drawbacks of Recursion in Data Structure

- Recursion uses stack space: Each recursive method call creates a new instance of the method, one with a brand new set of local variables. The complete stack space consumed is dependent upon the degree of nesting of this recursion method, and the number of local variables along with parameters.
- Recursion might carry out redundant computations: Look at the recursive computation of the Fibonacci sequence.

In total, an individual has to weigh the ease of the code presented by recursion against its downsides as outlined above. When a straightforward iterative alternative is achievable, it’s undoubtedly a much better option.

Towers of Hanoi

Input: The aim of the tower of Hanoi problem is to move the initial n different sized disks from needle A to needle C using a temporary needle B. The rule is that no larger disk is to be placed above the smaller disk in any of the needle while moving or at any time, and only the top of the disk is to be moved at a time from any needle to any needle.

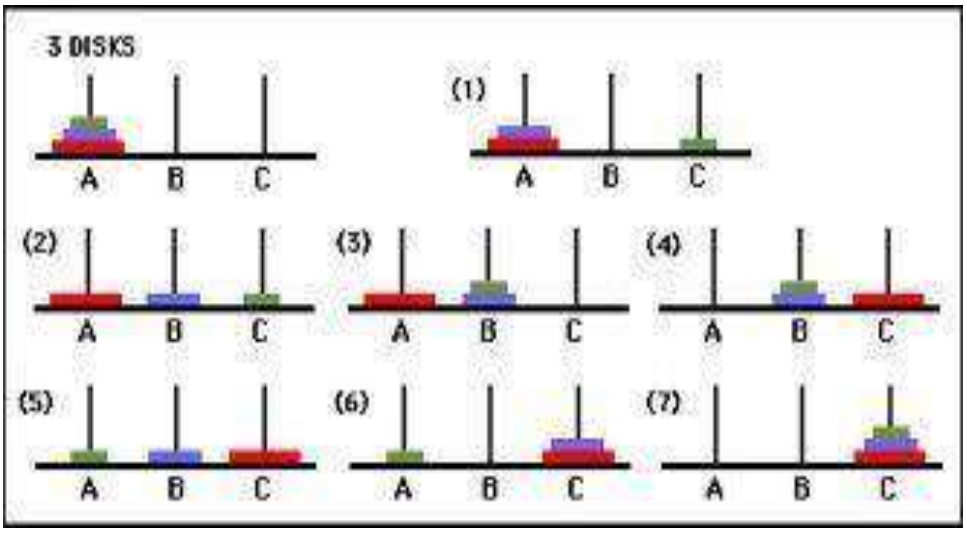
Output:

If n=1, move the single disk from A to C and return,

If n>1, move the top n-1 disks from A to B using C as temporary.

Move the remaining disk from A to C.

Move the n-1 disk disks from B to C, using A as temporary.



def TowerOfHanoi(n , from_rod, to_rod, aux_rod):
if n == 1:

print "Move disk 1 from rod",from_rod,"to rod",to_rod return

TowerOfHanoi(n-1, from_rod, aux_rod, to_rod)

print "Move disk",n,"from rod",from_rod,"to rod",to_rod TowerOfHanoi(n-1, aux_rod, to_rod, from_rod)

n = 4

TowerOfHanoi(n, 'A', 'C', 'B')